

TensorFlow.js：使用 TensorFlow.js 遷移學習技術，打造專屬的「訓練學習機器」

來源：<https://codelabs.developers.google.com/tensorflowjs-transfer-learning-teachable-machine?hl=zh-tw>

步驟數：17

在本程式碼研究室中

目錄

1. [事前準備](#)
2. [什麼是 TensorFlow.js ?](#)
3. [遷移學習](#)
4. [TensorFlow Hub - 基礎模型](#)
5. [設定程式碼](#)
6. [應用程式 HTML 樣板](#)
7. [新增樣式](#)
8. [JavaScript : 主要常數和監聽器](#)
9. [載入 MobileNet 基礎模型](#)
10. [定義新的模型頭](#)
11. [啟用網路攝影機](#)
12. [資料收集按鈕事件處理常式](#)
13. [資料收集](#)
14. [訓練與預測](#)
15. [實作重設按鈕](#)
16. [試試看吧 !](#)
17. [恭喜](#)

1. 事前準備

過去幾年來，TensorFlow.js 模型的使用量[呈指數成長](#)，許多 JavaScript 開發人員現在都想採用現有的頂尖模型，並重新訓練模型，使其能處理所屬產業特有的自訂資料。將現有模型 (通常稱為基礎模型) 用於類似但不同的領域，稱為遷移學習。

與從完全空白的模型開始訓練相比，遷移學習有許多優點。您可以重複使用先前訓練模型已學到的知識，而且只需要較少的分類新項目範例。此外，由於只需重新訓練模型架構的最後幾層，而非整個網路，因此訓練速度通常會大幅提升。因此，遷移學習非常適合用於網路瀏覽器環境，因為資源可能會因執行裝置而異，但也能直接存取感應器，輕鬆取得資料。

本程式碼實驗室將從空白畫布開始，逐步建構網頁應用程式，重現 Google 熱門的「[訓練學習機器](#)」網站。這個網站可讓您建立實用的網頁應用程式，使用者只要透過網路攝影機提供幾張範例圖片，就能辨識自訂物件。這個網站刻意保持簡潔，方便您專注於本程式碼研究室的機器學習部分。不過，與原始的訓練學習機器網站一樣，您有充分的空間運用現有的網頁程式開發人員經驗，改善使用者體驗。

必要條件

本程式碼實驗室適合對 TensorFlow.js 預先建構模型和基本 API 用法略有瞭解的網頁開發人員，協助他們開始使用 TensorFlow.js 進行遷移學習。

- 本實驗室假設您對 TensorFlow.js、HTML5、CSS 和 JavaScript 有基本瞭解。

如果您是 TensorFlow.js 的新手，建議先[參加這門免費的入門課程](#)，課程中會從基礎開始，逐步教導您機器學習和 TensorFlow.js 的相關知識。

課程內容

- 瞭解 TensorFlow.js，以及為何應在下一個網頁應用程式中使用這項技術。
- 如何建構簡化的 HTML/CSS /JavaScript 網頁，複製 訓練學習機器 的使用者體驗。

注意：我們未使用 React 或 Angular 等框架，但您可以視需要將網站調整為所選框架。

- 瞭解如何使用 TensorFlow.js 載入預先訓練的基礎模型 (具體來說是 MobileNet)，以產生可用於遷移學習的圖片特徵。
- 如何從使用者的網路攝影機收集資料，以辨識多個類別的資料。
- 如何建立及定義多層感知器，擷取圖片特徵並學習使用這些特徵分類新物件。

開始駭客任務吧！

軟硬體需求

- 建議使用 Glitch.com 帳戶，或您可自行編輯及執行的網路服務環境。
-

2. 什麼是 TensorFlow.js ?



TensorFlow.js 是開放原始碼機器學習程式庫，可在 JavaScript 執行的任何位置執行。這個程式庫是以原始的 [Python 版 TensorFlow 程式庫](#) 為基礎，目標是為 JavaScript 生態系統重新建立這種開發人員體驗和 API 集。

哪些類別可以套用顯示設定？

由於 JavaScript 具有可攜性，您現在只要使用 1 種語言，就能輕鬆在下列所有平台執行機器學習：

- 網頁瀏覽器中的用戶端，使用原生 JavaScript
- 伺服器端，甚至是 **Raspberry Pi** 等物聯網裝置 (使用 Node.js)
- 使用 Electron 的電腦應用程式
- 使用 React Native 的原生行動應用程式

TensorFlow.js 也支援這些環境中的多個後端 (例如 CPU 或 WebGL 等實際硬體環境)。在此情況下，「後端」並非指伺服器端環境，而是指執行後端 (例如 WebGL 中的用戶端)，確保相容性並維持快速運作。TensorFlow.js 目前支援：

- **在裝置的顯示卡 (GPU) 上執行 WebGL**：這是執行較大型模型 (超過 3 MB) 的最快方式，可透過 GPU 加速。
- **在 CPU 上執行 Web Assembly (WASM)** - 提升各種裝置的 CPU 效能，包括舊款手機。這類模型較小 (小於 3 MB)，由於將內容上傳至圖形處理器的負擔，這類模型在 CPU 上使用 WASM 執行的速度，實際上會比使用 WebGL 更快。
- **CPU 執行** - 如果沒有其他環境可用，則應採用這個備援方案。這是三者中最慢的，但隨時可供使用。

注意：如果您知道要執行的裝置，可以選擇[強制使用其中一個後端](#)，也可以不指定，讓 TensorFlow.js 為您決定。

用戶端超能力

在用戶端電腦的網路瀏覽器中執行 TensorFlow.js，可帶來多項值得考慮的優點。

隱私權

您可以在用戶端電腦上訓練及分類資料，完全不必將資料傳送至第三方網路伺服器。有時，這可能是為了遵守當地法律 (例如 GDPR) 的規定，或是處理使用者想保留在電腦上，不想傳送給第三方的資料。

速度

由於不必將資料傳送至遠端伺服器，推論 (分類資料的行為) 速度會更快。更棒的是，如果使用者授予存取權，您就能直接存取裝置的感應器，例如相機、麥克風、GPS、加速度計等。

觸及率和規模

只要按一下連結，世界各地的使用者就能在瀏覽器中開啟網頁，並使用您製作的內容。您不必為了使用機器學習系統，在伺服器端進行複雜的 Linux 設定，包括 CUDA 驅動程式等。

費用

由於沒有伺服器，您只需支付 CDN 費用，即可代管 HTML、CSS、JS 和模型檔案。與讓伺服器 (可能附有顯示卡) 全天候運作相比，CDN 的成本便宜許多。

伺服器端功能

運用 Node.js 實作的 TensorFlow.js 可啟用下列功能。

完整支援 CUDA

在伺服器端，如要使用顯示卡加速，必須安裝 [NVIDIA CUDA 驅動程式](#)，才能讓 TensorFlow 與顯示卡搭配運作 (與使用 WebGL 的瀏覽器不同，不需要安裝)。不過，如果完全支援 CUDA，就能充分運用顯示卡的低階功能，進而縮短訓練和推論時間。由於兩者共用相同的 C++ 後端，因此效能與 Python TensorFlow 實作項目相同。

模型大小

如果是研究領域的尖端模型，您可能會使用非常大的模型，大小可能達到 GB 級。由於每個瀏覽器分頁的記憶體用量有限制，目前無法在網路瀏覽器中執行這些模型。如要執行這些較大的模型，您可以在自己的伺服器上使用 Node.js，並具備有效執行這類模型所需的硬體規格。

IOT

Node.js 支援 Raspberry Pi 等熱門單板電腦，因此您也可以在这些裝置上執行 TensorFlow.js 模型。

速度

Node.js 是以 JavaScript 編寫而成，因此可從即時編譯中獲益。這表示使用 Node.js 時，您通常會發現效能有所提升，因為系統會在執行階段進行最佳化，特別是針對您可能執行的任何前置處理。如需這方面的絕佳範例，請參閱[這份個案研究](#)，瞭解 Hugging Face 如何使用 Node.js，將自然語言處理模型的效能提升一倍。

現在您已瞭解 TensorFlow.js 的基本概念、執行位置和一些優點，接下來就開始使用這項技術完成實用工作吧！

步驟 3 · 約 7 分鐘

3. 遷移學習

什麼是遷移學習？

遷移學習是指運用已學到的知識，協助學習其他類似的事物。

我們人類一直都是這樣。你的腦中儲存了畢生經驗，可用於辨識從未見過的新事物。以這棵柳樹為例：



視您所在地區而定，您可能從未見過這類樹木。

但如果我請你判斷下圖是否有柳樹，你可能很快就能找到，即使角度不同，且與我先前顯示的圖片略有差異。



您的大腦中已經有許多神經元，可辨識樹狀物體，也有其他神經元擅長找出長直線。你可以重複使用這項知識，快速分類柳樹。柳樹是一種樹狀物體，有許多長而直的垂直樹枝。

同樣地，如果您有已針對某個領域 (例如辨識圖片) 訓練的機器學習模型，可以重複使用該模型執行其他相關工作。

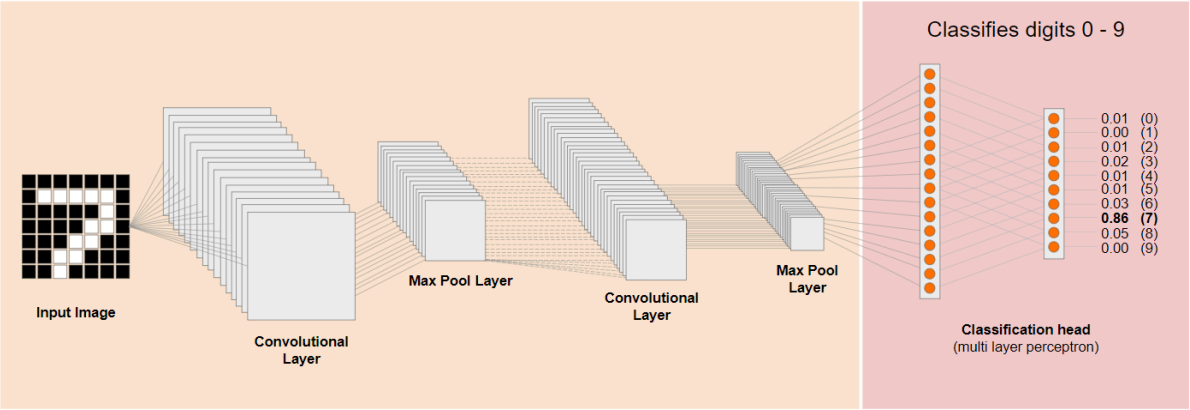
您也可以使用 MobileNet 等進階模型執行相同操作。MobileNet 是非常熱門的研究模型，可辨識 1000 種不同類型的物件。從狗到汽車，這項技術是透過名為 [ImageNet](#) 的龐大資料集訓練而成，其中包含數百萬張加上標籤的圖片。

在這部動畫中，您可以看到 MobileNet V1 模型中數量龐大的圖層：

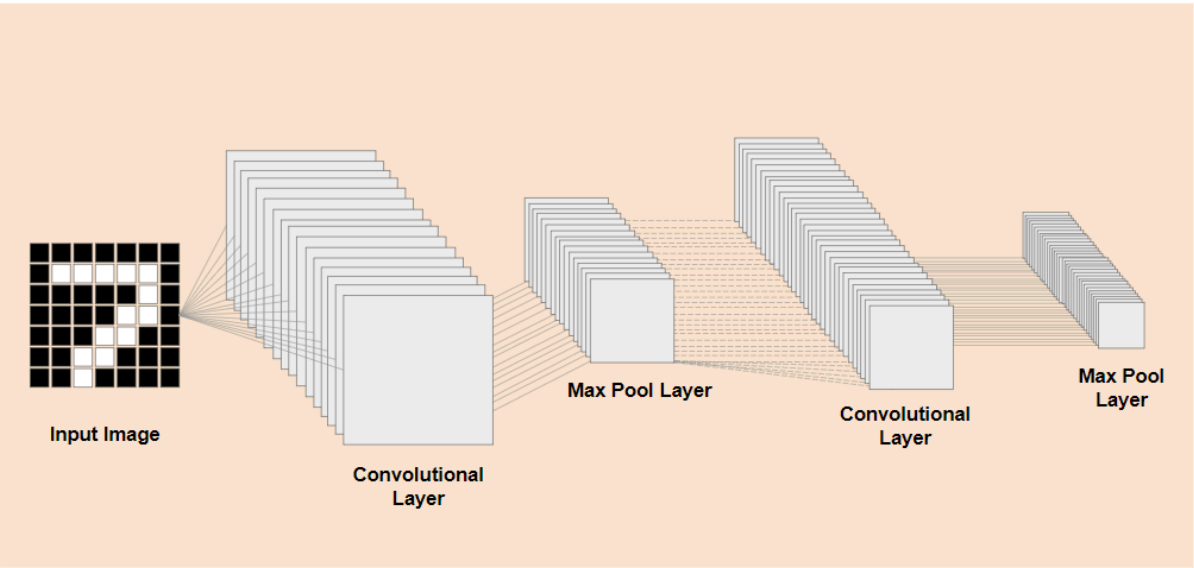
Layer (type)	Output shape	Param #
=====		
input_1 (InputLayer)	[null,224,224,3]	0
conv1 (Conv2D)	[null,112,112,8]	216
conv1_bn (BatchNormalization)	[null,112,112,8]	32
conv1_relu (Activation)	[null,112,112,8]	0
conv_dw_1 (DepthwiseConv2D)	[null,112,112,8]	72
conv_dw_1_bn (BatchNormalization)	[null,112,112,8]	32

在訓練過程中，這個模型學會如何擷取所有 1000 個物體的重要常見特徵，而模型用來辨識這類物體的許多低階特徵，也能用於偵測從未見過的新物體。畢竟，一切最終都只是線條、紋理和形狀的組合。

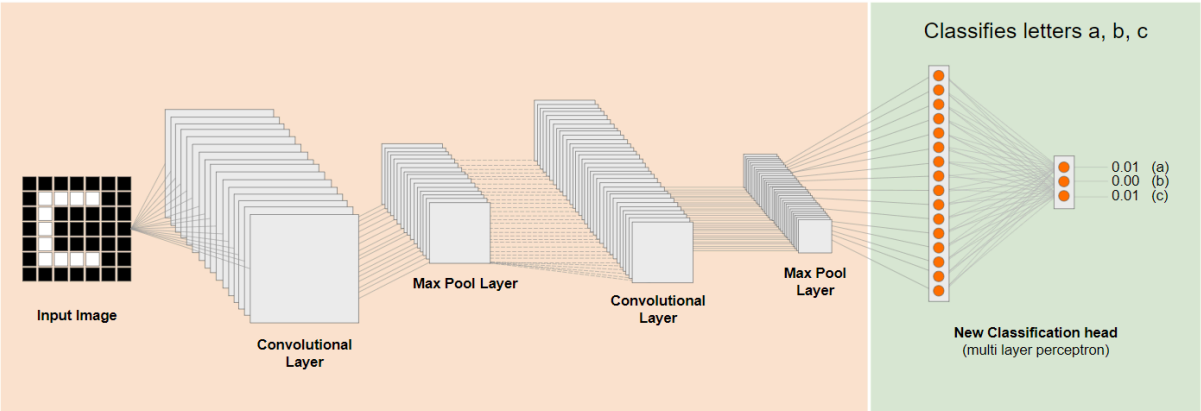
讓我們來看看傳統的卷積類神經網路 (CNN) 架構 (類似於 MobileNet)，瞭解遷移學習如何運用這個經過訓練的網路學習新事物。下圖顯示 CNN 的一般模型架構，在本例中，該模型經過訓練，可辨識手寫數字 0 到 9：



如左側所示，如果您可以將現有訓練模型的預先訓練低層級層，與模型尾端的分類層 (有時稱為模型的分類頭) 分開，就能使用低層級層，根據訓練時使用的原始資料，為任何指定圖片產生輸出特徵。以下是移除分類層的相同網路：



假設您嘗試辨識的新事物也能使用先前模型學到的這類輸出特徵，那麼這些特徵很有可能可以重複用於新用途。在上圖中，這個假設模型是根據數字訓練而成，因此學到的數字知識或許也能套用至字母 (例如 a、b 和 c)。因此，您現在可以新增分類標頭，嘗試預測 a、b 或 c，如下所示：



這裡的較低層級會凍結，不會接受訓練，只有新的分類標題會自行更新，從左側預先訓練的切碎模型提供的特徵中學習。這項作業稱為「遷移學習」，也是 Teachable Machine 在幕後執行的作業。

您也可以看到，由於只需要在網路的最後訓練多層感知器，因此訓練速度比從頭訓練整個網路快得多。

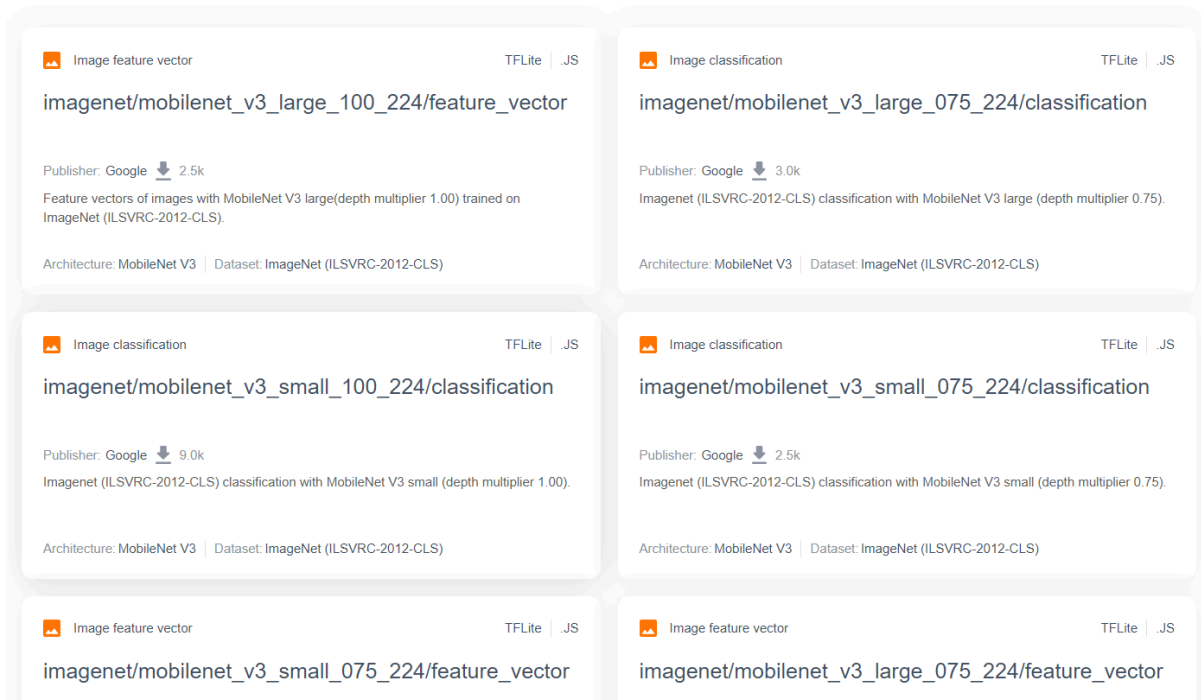
但如何取得模型的子部分？如要瞭解詳情，請參閱下一節。

步驟 4 · 約 5 分鐘

4. TensorFlow Hub - 基礎模型

尋找合適的基礎模型

如要使用 MobileNet 等更進階且熱門的研究模型，請前往 [TensorFlow Hub](#)，然後篩選適用於 [TensorFlow.js](#) 且使用 [MobileNet v3 架構的模型](#)，即可找到如下所示的結果：



請注意，部分結果屬於「圖片分類」類型 (詳見每個模型資訊卡結果的左上角)，其他則屬於「圖片特徵向量」類型。

這些圖片特徵向量結果基本上是 MobileNet 的預先切分版本，可用於取得圖片特徵向量，而非最終分類。

這類模型通常稱為「基礎模型」，您可以新增分類標題，並使用自己的資料訓練模型，以執行遷移學習，方法與前一節所示相同。

接下來要檢查的是，對於感興趣的特定基礎模型，該模型是以哪種 TensorFlow.js 格式發布。如果您開啟其中一個特徵向量 MobileNet v3 模型頁面，可以從 JS 說明文件中看到，這是以說明文件中的範例程式碼片段為基礎的圖形模型，並使用

```
tf.loadGraphModel()
```

Inputs

The input `images` are expected to have color values in the range [0,1], following the [common image input](#) conventions. For this model `height` x `width` = 224 x 224 pixels.

Example use

This model can be used with [TensorFlow.js](#) as:

```
// Be sure to load TensorFlow.js on your page. See
// https://github.com/tensorflow/tfjs#getting-started.

const model = await tf.loadGraphModel(
  'https://tfhub.dev/google/tfjs-model/imagenet/mobilenet_v3_small_100_224/feature_vector/5/default/1',
  { fromTFHub: true });
```

此外，如果模型採用的是層格式而非圖格式，您可以選擇要凍結哪些層，以及要取消凍結哪些層以進行訓練。為新工作建立模型時，這項功能非常實用，通常稱為「轉移模型」。不過，目前您會使用本教學課程的預設圖形模型類型，因為大多數 TF Hub 模型都是以這種形式部署。如要進一步瞭解如何使用 Layers 模型，請參閱 [TensorFlow.js 零基礎到高手](#) 課程。

遷移學習的優點

相較於從頭開始訓練整個模型架構，使用遷移學習有什麼優點？

首先，使用遷移學習方法的主要優勢在於訓練時間，因為您已有訓練好的基礎模型可供建構。

其次，由於已完成訓練，您只需顯示少數幾個要分類的新事物範例即可。

如果您收集分類目標的範例資料時，時間和資源有限，而且需要快速製作原型，再收集更多訓練資料來強化模型，這項功能就非常實用。

由於遷移學習需要較少的資料，且訓練較小網路的速度較快，因此資源密集程度較低。因此非常適合瀏覽器環境，在現代機器上只需幾十秒，而不是數小時、數天或數週，即可完成完整模型訓練。

提示：如要對複雜網路 (例如深層 CNN) 執行完整模型訓練，伺服器端的 Node.js 是較好的選擇。您可以在這裡使用 TensorFlow.js Node，更有效率地訓練模型，產生可在瀏覽器中快速執行的模型，因為推論作業已針對用戶端進行高度最佳化。

好的！現在您已瞭解遷移學習的本質，可以開始建立自己的訓練學習機器版本。立即開始！

5. 設定程式碼

軟硬體需求

- 新式網路瀏覽器。
- 具備 HTML、CSS、JavaScript 和 [Chrome 開發人員工具](#) (查看控制台輸出內容) 的基本知識。

開始編寫程式碼

我們已為 [Glitch.com](#) 或 [Codepen.io](#) 建立可做為起點的樣板範本。您只需按一下滑鼠，即可複製任一範本，做為本程式碼研究室的基礎狀態。

在 Glitch 上，按一下「remix this」按鈕來建立分支，並製作可編輯的新檔案集。

或者，在 Codepen 上，按一下畫面右下角的「fork」。

這個非常簡單的架構會提供下列檔案：

- HTML 網頁 (index.html)
- 樣式表 (style.css)
- 用於編寫 JavaScript 程式碼的檔案 (script.js)

為方便起見，HTML 檔案中已新增 TensorFlow.js 程式庫的匯入項目。這是訂閱按鈕的圖示：

index.html

```
<!-- Import TensorFlow.js library -->
<script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs/dist/tf.min.js"
type="text/javascript"></script>
```

替代做法：使用偏好的網頁編輯器或在本機工作

如要下載程式碼並在本機或使用其他線上編輯器作業，只要在相同目錄中建立上述 3 個檔案，然後將 Glitch 樣板中的程式碼複製並貼到每個檔案即可。

注意：如果您是在本機執行，而非 Codepen.io 或類似網站，請務必透過網路伺服器提供檔案，而非只是按兩下 index.html，透過 file:// 通訊協定開啟檔案。

步驟 6 · 約 3 分鐘

6. 應用程式 HTML 樣板

該從何著手？

所有原型都需要一些基本的 HTML 支架，您可以在其中呈現發現。請立即設定。您將新增：

- 頁面標題。
- 一些說明文字。
- 狀態段落。
- 準備就緒後，用來保留網路攝影機動態饋給的影片。
- 多個按鈕，可啟動相機、收集資料或重設體驗。
- 匯入 TensorFlow.js 和稍後要編寫的 JS 檔案。

開啟 `index.html`，然後貼上下列程式碼，覆寫現有程式碼，即可設定上述功能：

`index.html`

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Transfer Learning - TensorFlow.js</title>
    <meta charset="utf-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <!-- Import the webpage's stylesheet -->
    <link rel="stylesheet" href="/style.css">
  </head>
  <body>
    <h1>Make your own "Teachable Machine" using Transfer Learning with MobileNet v3 in TensorFlow.js using saved graph model from TFHub.</h1>

    <p id="status">Awaiting TF.js load</p>

    <video id="webcam" autoplay muted></video>

    <button id="enableCam">Enable Webcam</button>
    <button class="dataCollector" data-lhot="0" data-name="Class 1">Gather Class 1
Data</button>
    <button class="dataCollector" data-lhot="1" data-name="Class 2">Gather Class 2
Data</button>
    <button id="train">Train & Predict!</button>
    <button id="reset">Reset</button>

    <!-- Import TensorFlow.js library -->
    <script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@3.11.0/dist/tf.min.js"
type="text/javascript"></script>

    <!-- Import the page's JavaScript to do some stuff -->
    <script type="module" src="/script.js"></script>
  </body>
</html>

```

細分

讓我們分解上述部分 HTML 程式碼，重點說明您新增的內容。

- 您已為網頁標題新增 `<h1>` 標記，以及 ID 為「status」的 `<p>` 標記，您將在該處列印資訊，以便使用系統的不同部分查看輸出內容。
- 您新增了 ID 為「webcam」的 `<video>` 元素，稍後會將網路攝影機串流算繪至該元素。
- 您新增了 5 個 `<button>` 元素。第一個 ID 為「enableCam」，可啟用攝影機。接下來的兩個按鈕的類別為「dataCollector」，可讓您收集要辨識的物件範例圖片。稍後編寫的程式碼會經過設計，因此您可以新增任意數量的按鈕，這些按鈕會自動如預期運作。

請注意，這些按鈕也有名為 `data-lhot` 的特殊使用者定義屬性，第一個類別的整數值從 0 開始。這是用來表示特定類別資料的數值索引。由於機器學習模型只能處理數字，因此索引會用於以數字表示而非字串，正確編碼輸出類別。

此外，還有 `data-name` 屬性，其中包含您要用於這個類別的易讀名稱，讓您為使用者提供更有意義的名稱，而不是 1 熱編碼中的數值索引值。

最後，您可以使用「訓練」和「重設」按鈕，在收集資料後啟動訓練程序，或重設應用程式。

- 您也新增了 2 個 `<script>` 匯入項目。一個用於 TensorFlow.js，另一個用於您稍後定義的 script.js。
-

7. 新增樣式

元素預設值

為剛新增的 HTML 元素新增樣式，確保這些元素能正確顯示。以下是新增的樣式，可正確設定元素的位置和大小。沒什麼特別的。您當然可以稍後再新增內容，進一步提升使用者體驗，就像在 Teachable Machine 影片中看到的那樣。

style.css

```
body {  
  font-family: helvetica, arial, sans-serif;  
  margin: 2em;  
}
```

```
h1 {  
  font-style: italic;  
  color: #FF6F00;  
}
```

```
video {  
  clear: both;  
  display: block;  
  margin: 10px;  
  background: #000000;  
  width: 640px;  
  height: 480px;  
}
```

```
button {  
  padding: 10px;  
  float: left;  
  margin: 5px 3px 5px 10px;  
}
```

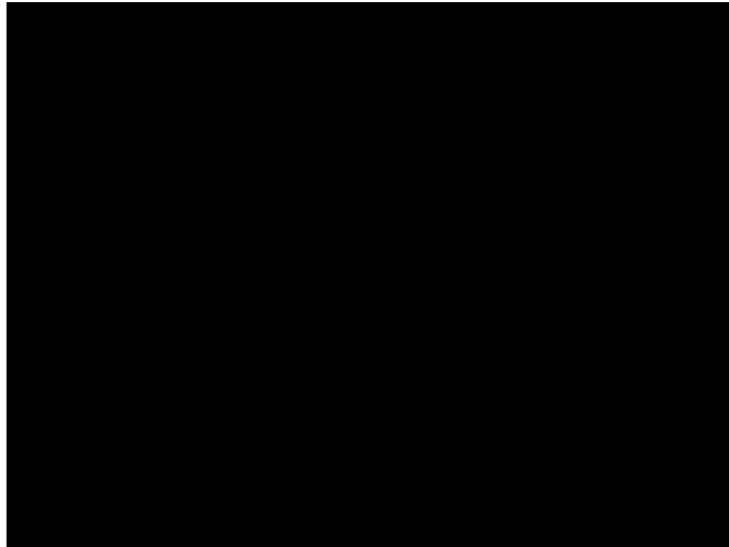
```
.removed {  
  display: none;  
}
```

```
#status {  
  font-size: 150%;  
}
```

太好了！就是這麼簡單。如果現在預覽輸出內容，畫面應如下所示：

Make your own "Teachable Machine" using Transfer Learning with MobileNet v3 in TensorFlow.js using saved graph model from TFHub.

Awaiting TF.js load



Enable Webcam	Gather Class 1 Data	Gather Class 2 Data	Train & Predict!	Reset
---------------	---------------------	---------------------	------------------	-------

提示：如要在 [Glitch](#) 中預覽即時作業，請點選畫面左上方的「show」下拉式選單，然後選取「next to the code」或「in a new window」。

8. JavaScript：主要常數和監聽器

提示：下方程式碼依序顯示，但為了深入說明，我們將其分成較小的部分。每次都將每個區段複製並貼到 JavaScript 檔案結尾，或是取代上一個步驟中定義的空白函式，一切應該都能正常運作。

定義重要常數

首先，請新增一些您會在整個應用程式中使用的重要常數。請先將 `script.js` 的內容替換為下列常數：

`script.js`

```
const STATUS = document.getElementById('status');
const VIDEO = document.getElementById('webcam');
const ENABLE_CAM_BUTTON = document.getElementById('enableCam');
const RESET_BUTTON = document.getElementById('reset');
const TRAIN_BUTTON = document.getElementById('train');
const MOBILE_NET_INPUT_WIDTH = 224;
const MOBILE_NET_INPUT_HEIGHT = 224;
const STOP_DATA_GATHER = -1;
const CLASS_NAMES = [];
```

以下說明這些用途：

- `STATUS` 只是保留對段落標記的參照，您將在其中撰寫狀態更新。
- `VIDEO` 會保留 HTML 影片元素的參照，該元素將算繪網路攝影機畫面。
- `ENABLE_CAM_BUTTON`、`RESET_BUTTON` 和 `TRAIN_BUTTON` 會從 HTML 網頁擷取所有重要按鈕的 DOM 參照。
- `MOBILE_NET_INPUT_WIDTH` 和 `MOBILE_NET_INPUT_HEIGHT` 分別定義 MobileNet 模型的預期輸入寬度和高度。將這個值儲存在檔案頂端的常數中，日後如果決定使用不同版本，只要更新一次值即可，不必在許多不同位置替換。
- `STOP_DATA_GATHER` 設為 -1。這會儲存狀態值，讓您瞭解使用者何時停止點選按鈕，以便從網路攝影機動態饋給收集資料。為這個數字提供更有意義的名稱，可讓程式碼更容易閱讀。
- `CLASS_NAMES` 會做為查閱表，並保留可能類別預測結果的人類可讀名稱。這個陣列稍後會填入資料。

好了，現在您已參照重要元素，接下來要將一些事件監聽器與這些元素建立關聯。

新增重要事件監聽器

首先，請為主要按鈕新增點擊事件處理常式，如下所示：

`script.js`

```

ENABLE_CAM_BUTTON.addEventListener('click', enableCam);
TRAIN_BUTTON.addEventListener('click', trainAndPredict);
RESET_BUTTON.addEventListener('click', reset);

function enableCam() {
  // TODO: Fill this out later in the codelab!
}

function trainAndPredict() {
  // TODO: Fill this out later in the codelab!
}

function reset() {
  // TODO: Fill this out later in the codelab!
}

```

`ENABLE_CAM_BUTTON`：點選時會呼叫 `enableCam` 函式。

`TRAIN_BUTTON` - 點選時呼叫 `trainAndPredict`。

`RESET_BUTTON` - 點選時會重設通話。

最後，您可以使用 `document.querySelectorAll()` 找出這個部分中，所有類別為「`dataCollector`」的按鈕。這會傳回從文件中找到的元素陣列，這些元素符合下列條件：

script.js

```

let dataCollectorButtons = document.querySelectorAll('button.dataCollector');
for (let i = 0; i < dataCollectorButtons.length; i++) {
  dataCollectorButtons[i].addEventListener('mousedown', gatherDataForClass);
  dataCollectorButtons[i].addEventListener('mouseup', gatherDataForClass);
  // Populate the human readable names for classes.
  CLASS_NAMES.push(dataCollectorButtons[i].getAttribute('data-name'));
}

function gatherDataForClass() {
  // TODO: Fill this out later in the codelab!
}

```

程式碼說明：

接著，您會逐一檢查找到的按鈕，並將 2 個事件監聽器與每個按鈕建立關聯。一個用於「`mousedown`」，另一個用於「`mouseup`」。只要按住按鈕，就能持續錄製樣本，方便收集資料。

這兩個事件都會呼叫稍後定義的 `gatherDataForClass` 函式。

警告：本程式碼研究室著重於機器學習，並非使用者介面設計課程，因此請注意，使用這些按鈕時，您必須將滑鼠游標停留在按鈕上，才能同時擷取「mousedown」和「mouseup」事件。如果您點選按鈕，然後將滑鼠移至使用者介面中的其他位置 (按鈕區域外)，再放開按鈕，按鈕就不會觸發 `mouseup`，導致系統永久收集資料。這個極端情況可以透過一些額外邏輯修正，但超出本程式碼研究室的範圍，因此這裡的程式碼會盡量簡潔。

此時，您也可以將 HTML 按鈕屬性 `data-name` 中找到的人類可讀類別名稱，推送至 `CLASS_NAMES` 陣列。

提示：在此使用類別尋找按鈕，而非硬式編碼固定數量的「dataCollector」按鈕，之後您就能變更 HTML 來新增更多按鈕，而程式碼仍可正確運作，適用於 3 個、4 個或任意數量的按鈕。

接著，新增一些變數，用於儲存稍後會用到的重要項目。

script.js

```
let mobilenet = undefined;
let gatherDataState = STOP_DATA_GATHER;
let videoPlaying = false;
let trainingDataInputs = [];
let trainingDataOutputs = [];
let examplesCount = [];
let predict = false;
```

我們來看看這些步驟。

首先，您有一個變數 `mobilenet`，用於儲存載入的 `mobilenet` 模型。一開始請將此值設為未定義。

接著，您會看到名為 `gatherDataState` 的變數。如果按下「dataCollector」按鈕，系統會改為顯示該按鈕的 1 熱 ID (如 HTML 中所定義)，方便您瞭解當下收集的資料類別。一開始，這會設為 `STOP_DATA_GATHER`，這樣您稍後編寫的資料收集迴圈就不會在沒有按下任何按鈕時收集資料。

`videoPlaying` 會追蹤網路攝影機串流是否已成功載入及播放，並可供使用。一開始，這會設為 `false`，因為你按下 `ENABLE_CAM_BUTTON` 前，網路攝影機不會開啟。

接著，定義 2 個陣列 `trainingDataInputs` 和 `trainingDataOutputs`。當您點擊 `MobileNet` 基礎模型產生的輸入特徵和取樣輸出類別的「dataCollector」按鈕時，這些會儲存收集到的訓練資料值。

最後，系統會定義 `examplesCount`，陣列，追蹤您開始新增範例後，每個類別包含的範例數量。

最後，您有一個名為 `predict` 的變數，可控制預測迴圈。一開始會設為 `false`。如未設定為「稍後」`true`，系統將無法進行預測。

現在所有重要變數都已定義完畢，接下來請載入預先切分的 `MobileNet v3` 基本模型，這個模型會提供圖片特徵向量，而非分類。

步驟 9 · 約 7 分鐘

9. 載入 MobileNet 基礎模型

首先，定義名為 `loadMobileNetFeatureModel` 的新函式，如下所示。這必須是非同步函式，因為載入模型是屬於非同步作業：

script.js

```
/**
 * Loads the MobileNet model and warms it up so ready for use.
 */
async function loadMobileNetFeatureModel() {
  const URL =
    'https://tfhub.dev/google/tfjs-
model/imagenet/mobilenet_v3_small_100_224/feature_vector/5/default/1';

  mobilenet = await tf.loadGraphModel(URL, {fromTFHub: true});
  STATUS.innerText = 'MobileNet v3 loaded successfully!';

  // Warm up the model by passing zeros through it once.
  tf.tidy(function () {
    let answer = mobilenet.predict(tf.zeros([1, MOBILE_NET_INPUT_HEIGHT,
MOBILE_NET_INPUT_WIDTH, 3]));
    console.log(answer.shape);
  });
}

// Call the function immediately to start loading.
loadMobileNetFeatureModel();
```

在這段程式碼中，您會定義 `URL`，也就是 TFHub 說明文件中模型載入的位置。

然後使用 `await tf.loadGraphModel()` 載入模型，並記得在從這個 Google 網站載入模型時，將特殊屬性 `fromTFHub` 設為 `true`。這項特殊情況僅適用於使用 TF Hub 上託管的模型，且必須設定這項額外屬性。

載入完成後，您可以在 `STATUS` 元素的 `innerText` 中設定訊息，以便查看是否已正確載入，並準備開始收集資料。

現在只要暖機模型即可。使用這類大型模型時，第一次使用模型時可能需要一些時間設定所有項目。因此，建議您將零傳遞至模型，避免日後在時間更為重要的情況下發生等待。

您可以使用以 `tf.tidy()` 包裝的 `tf.zeros()`，確保張量正確處置，批量大小為 1，且高度和寬度正確無誤，與您在開頭定義的常數相同。最後，您也會指定色彩通道，在本例中為 3，因為模型預期會收到 RGB 圖片。

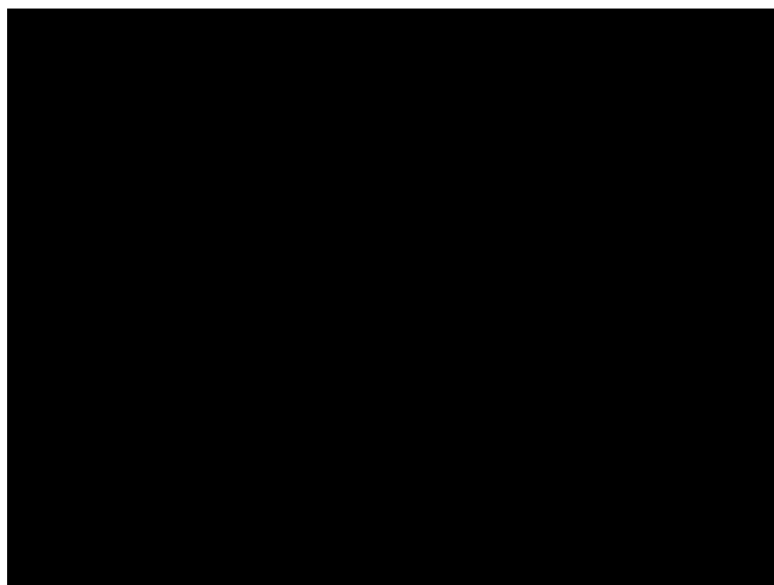
接著，使用 `answer.shape()` 記錄傳回的張量形狀，協助您瞭解這個模型產生的圖像特徵大小。

定義此函式後，您就能立即呼叫函式，在載入網頁時啟動模型下載作業。

如果現在查看即時預覽畫面，過一會兒，狀態文字就會從「Awaiting TF.js load」變成「MobileNet v3 loaded successfully!」，如下所示。請先確認這項功能正常運作，再繼續操作。

Make your own "Teachable Machine" using Transfer Learning with MobileNet v3 in TensorFlow.js using saved graph model from TFHub.

MobileNet v3 loaded successfully!



Enable Webcam	Gather Class 1 Data	Gather Class 2 Data	Train & Predict!	Reset
---------------	---------------------	---------------------	------------------	-------

您也可以查看控制台輸出內容，瞭解模型產生的輸出特徵列印大小。將零值傳遞至 MobileNet 模型後，您會看到列印的 `[1, 1024]` 形狀。第一個項目只是大小為 1 的批量，您會發現系統實際傳回 1024 個特徵，可用於分類新物件。

10. 定義新的模型頭

現在要定義模型標頭，這基本上是極簡的多層感知器。

script.js

```
let model = tf.sequential();
model.add(tf.layers.dense({inputShape: [1024], units: 128, activation: 'relu'}));
model.add(tf.layers.dense({units: CLASS_NAMES.length, activation: 'softmax'}));

model.summary();

// Compile the model with the defined optimizer and specify a loss function to use.
model.compile({
  // Adam changes the learning rate over time which is useful.
  optimizer: 'adam',
  // Use the correct loss function. If 2 classes of data, must use binaryCrossentropy.
  // Else categoricalCrossentropy is used if more than 2 classes.
  loss: (CLASS_NAMES.length === 2) ? 'binaryCrossentropy': 'categoricalCrossentropy',
  // As this is a classification problem you can record accuracy in the logs too!
  metrics: ['accuracy']
});
```

讓我們逐步瞭解這段程式碼。首先，請定義 `tf.sequential` 模型，並在其中加入模型層。

接著，將稠密層新增為這個模型的輸入層。輸入形狀為 `1024`，因為 MobileNet v3 特徵的輸出大小就是這樣。您在上一個步驟中將 `1` 傳遞至模型後，發現了這點。這個層有 128 個神經元，使用 ReLU 活化函數。

如果您不熟悉啟動函式和模型層，建議[參加本研討會開頭詳述的課程](#)，瞭解這些屬性在幕後的作用。

接下來要新增的層是輸出層。神經元數量應等於您要預測的類別數量。如要這麼做，請使用 `CLASS_NAMES.length` 找出您打算分類的類別數量，這等於使用者介面中的資料收集按鈕數量。由於這是分類問題，因此您要在這個輸出層使用 `softmax` 啟動，嘗試建立模型來解決分類問題時，必須使用這個啟動，而非迴歸。

現在請列印 `model.summary()`，將新定義模型的總覽列印到控制台。

最後，編譯模型，準備進行訓練。這裡的最佳化工具設為 `adam`，如果 `CLASS_NAMES.length` 等於 `2`，損失會是 `binaryCrossentropy`，如果需要分類的類別有 3 個以上，則會使用 `categoricalCrossentropy`。系統也會要求提供準確度指標，以便稍後在記錄中監控，用於偵錯。

控制台應會顯示類似下列的內容：

Layer (type)	Output shape	Param #
=====		
dense_Dense1 (Dense)	[null,128]	131200
=====		
dense_Dense2 (Dense)	[null,2]	258
=====		
Total params: 131458		
Trainable params: 131458		
Non-trainable params: 0		

請注意，這項模型有超過 13 萬個可訓練參數。但由於這是由一般神經元組成的簡單稠密層，因此訓練速度相當快。

專案完成後，您可以嘗試變更第一層的神經元數量，看看在仍能獲得良好效能的情況下，神經元數量可以降到多低。通常，機器學習需要經過一定程度的試錯，才能找出最佳參數值，在資源用量和速度之間取得最佳平衡。

11. 啟用網路攝影機

現在請充實先前定義的 `enableCam()` 函式。新增名為 `hasGetUserMedia()` 的函式 (如下所示)，然後將先前定義的 `enableCam()` 函式內容替換為下列對應程式碼。

script.js

```
function hasGetUserMedia() {
  return !(navigator.mediaDevices && navigator.mediaDevices.getUserMedia);
}

function enableCam() {
  if (hasGetUserMedia()) {
    // getUsermedia parameters.
    const constraints = {
      video: true,
      width: 640,
      height: 480
    };

    // Activate the webcam stream.
    navigator.mediaDevices.getUserMedia(constraints).then(function(stream) {
      VIDEO.srcObject = stream;
      VIDEO.addEventListener('loadeddata', function() {
        videoPlaying = true;
        ENABLE_CAM_BUTTON.classList.add('removed');
      });
    });
  } else {
    console.warn('getUserMedia() is not supported by your browser');
  }
}
```

首先，請建立名為 `hasGetUserMedia()` 的函式，檢查主要瀏覽器 API 屬性是否存在，藉此確認瀏覽器是否支援 `getUserMedia()`。

在 `enableCam()` 函式中，使用您剛才定義的 `hasGetUserMedia()` 函式，檢查是否支援。如果不是，請在控制台中列印警告。

如果支援，請為 `getUserMedia()` 呼叫定義一些限制，例如只想要影片串流，以及偏好影片的 `width` 大小為 640 像素，`height` 大小為 480 像素。這是因為因為影片必須調整為 224 x 224 像素，才能輸入至 MobileNet 模型，因此影片大於這個尺寸就沒有意義。要求較小的解析度，也能節省一些運算資源。大多數攝影機都支援這個大小的解析度。

接著，使用上述 `constraints` 呼叫 `navigator.mediaDevices.getUserMedia()`，然後等待傳回 `stream`。傳回 `stream` 後，您可以將其設為 `VIDEO` 元素的 `srcObject` 值，讓 `VIDEO` 元素播放 `stream`。

您也應在 `VIDEO` 元素上新增 `eventListener`，瞭解 `stream` 何時載入並順利播放。

載入串流後，您可以將 `videoPlaying` 設為 `true`，並移除 `ENABLE_CAM_BUTTON`，方法是將其類別設為「`removed`」，防止使用者再次點選。

現在請執行程式碼，按一下「啟用攝影機」按鈕，然後允許存取網路攝影機。如果是第一次執行這項操作，您應該會看到自己算繪到網頁的影片元素，如下所示：

Make your own "Teachable Machine" using Transfer Learning with MobileNet v3 in TensorFlow.js using saved graph model from TFHub.

MobileNet v3 loaded successfully!



Gather Class 1 Data

Gather Class 2 Data

Train & Predict!

Reset

好了，現在要新增函式來處理 `dataCollector` 按鈕點擊事件。

12. 資料收集按鈕事件處理常式

現在請填寫目前空白的 `gatherDataForClass()` 函式。這是您在程式碼研究室開始時，為 `dataCollector` 按鈕指派的事件處理常式函式。

script.js

```
/**
 * Handle Data Gather for button mouseup/mousedown.
 */
function gatherDataForClass() {
  let classNumber = parseInt(this.getAttribute('data-lhot'));
  gatherDataState = (gatherDataState === STOP_DATA_GATHER) ? classNumber : STOP_DATA_GATHER;
  dataGatherLoop();
}
```

首先，請以屬性名稱 (本例中為 `data-lhot`) 做為參數呼叫 `this.getAttribute()`，檢查目前點選按鈕的 `data-lhot` 屬性。由於這是字串，因此您可以使用 `parseInt()` 將其轉換為整數，並將結果指派給名為 `classNumber` 的變數

接著，請視情況設定 `gatherDataState` 變數。如果目前的 `gatherDataState` 等於 `STOP_DATA_GATHER` (您已設為 -1)，表示您目前未收集任何資料，且觸發的是 `mousedown` 事件。將 `gatherDataState` 設為您剛找到的 `classNumber`。

否則，這表示您目前正在收集資料，且觸發的事件為 `mouseup` 事件，而您現在想停止收集該類別的資料。只要將其設回 `STOP_DATA_GATHER` 狀態，即可結束稍後定義的資料收集迴圈。

最後，啟動對 `dataGatherLoop()` 的呼叫，實際執行類別資料的記錄作業。

13. 資料收集

現在請定義 `dataGatherLoop()` 函數。這個函式負責從網路攝影機影片中取樣圖片、透過 MobileNet 模型傳遞圖片，並擷取該模型的輸出內容 (1024 個特徵向量)。

接著，系統會儲存這些資料，以及目前按下的按鈕 `gatherDataState` ID，方便您瞭解這些資料代表的類別。

以下將逐步說明：

script.js

```
function dataGatherLoop() {
  if (videoPlaying && gatherDataState !== STOP_DATA_GATHER) {
    let imageFeatures = tf.tidy(function() {
      let videoFrameAsTensor = tf.browser.fromPixels(VIDEO);
      let resizedTensorFrame = tf.image.resizeBilinear(videoFrameAsTensor,
[MOBILE_NET_INPUT_HEIGHT,
      MOBILE_NET_INPUT_WIDTH], true);
      let normalizedTensorFrame = resizedTensorFrame.div(255);
      return mobilenet.predict(normalizedTensorFrame.expandDims()).squeeze();
    });

    trainingDataInputs.push(imageFeatures);
    trainingDataOutputs.push(gatherDataState);

    // Initialize array index element if currently undefined.
    if (examplesCount[gatherDataState] === undefined) {
      examplesCount[gatherDataState] = 0;
    }
    examplesCount[gatherDataState]++;

    STATUS.innerText = '';
    for (let n = 0; n < CLASS_NAMES.length; n++) {
      STATUS.innerText += CLASS_NAMES[n] + ' data count: ' + examplesCount[n] + ' . ';
    }
    window.requestAnimationFrame(dataGatherLoop);
  }
}
```

只有在 `videoPlaying` 為 `true` (表示網路攝影機已啟用)、`gatherDataState` 不等於 `STOP_DATA_GATHER`，且目前正在按下類別資料收集按鈕時，您才會繼續執行這項函式。

接著，將程式碼包裝在 `tf.tidy()` 中，以便在後續程式碼中處置所有建立的張量。這段 `tf.tidy()` 程式碼的執行結果會儲存在名為 `imageFeatures` 的變數中。

您現在可以使用 `tf.browser.fromPixels()` 擷取網路攝影機 `VIDEO` 的畫面。含有圖片資料的結果張量會儲存在名為 `videoFrameAsTensor` 的變數中。

接著，將 `videoFrameAsTensor` 變數調整為 MobileNet 模型輸入內容的正確形狀。使用 `tf.image.resizeBilinear()` 呼叫，並將要重塑形狀的張量做為第一個參數，然後將定義新高度和寬度的形狀做為第二個參數，如先前建立的常數所定義。最後，請將 `align corners` 設為 `true`，並傳遞第三個參數，以免在調整大小時發生對齊問題。縮放結果會儲存在名為 `resizedTensorFrame` 的變數中。

請注意，由於網路攝影機圖片大小為 640 x 480 像素，而模型需要 224 x 224 像素的正方形圖片，因此這個原始大小調整會拉伸圖片。

就本示範而言，這應該沒問題。不過，完成本程式碼研究室後，您可能會想嘗試從這張圖片裁剪出正方形，以便在日後建立的任何生產系統中獲得更出色的結果。

接著，將圖片資料正規化。使用 `tf.browser.frompixels()` 時，圖像資料一律介於 0 到 255 之間，因此您可以直接將 `resizedTensorFrame` 除以 255，確保所有值都介於 0 到 1 之間，這也是 MobileNet 模型預期的輸入內容。

最後，在程式碼的 `tf.tidy()` 部分，呼叫 `mobilenet.predict()` 並傳遞使用 `expandDims()` 擴展的 `normalizedTensorFrame`，將這個標準化張量推送至載入的模型，使其成為大小為 1 的批次，因為模型預期會收到一批輸入內容進行處理。

結果傳回後，您可以立即對該傳回的結果呼叫 `squeeze()`，將其壓縮回 1D 張量，然後傳回並指派給 `imageFeatures` 變數，擷取 `tf.tidy()` 的結果。

現在您已取得 MobileNet 模型中的 `imageFeatures`，可以將這些資料推送至先前定義的 `trainingDataInputs` 陣列，藉此記錄這些資料。

您也可以將目前的 `gatherDataState` 推送到 `trainingDataOutputs` 陣列，記錄這個輸入內容代表的意義。

請注意，在先前定義的 `gatherDataForClass()` 函式中，當按鈕遭到點選時，`gatherDataState` 變數會設為您要記錄資料的目前類別數值 ID。

此時，您也可以增加特定類別的範例數量。如要這麼做，請先檢查 `examplesCount` 陣列中的索引是否已初始化。如果未定義，請將其設為 0，初始化特定類別數值 ID 的計數器，然後遞增目前 `gatherDataState` 的 `examplesCount`。

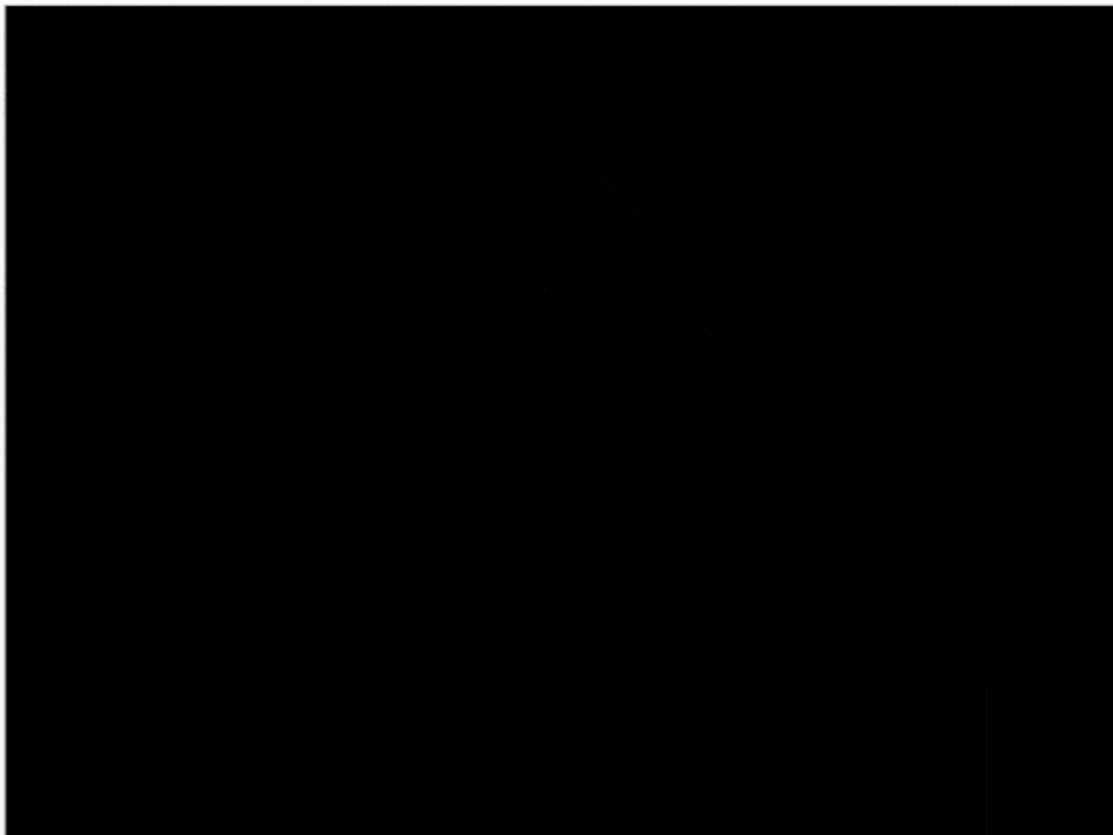
現在請更新網頁上 `STATUS` 元素的文字，顯示擷取的每個類別目前計數。如要這麼做，請在 `CLASS_NAMES` 陣列中執行迴圈，並列印可讀取的名稱，以及 `examplesCount` 中相同索引的資料計數。

最後，呼叫 `window.requestAnimationFrame()` 並傳遞 `dataGatherLoop` 做為參數，以遞迴方式再次呼叫這個函式。系統會持續從影片中取樣影格，直到偵測到按鈕的 `mouseup`，並將 `gatherDataState` 設為 `STOP_DATA_GATHER`，為止，此時資料收集迴圈就會結束。

現在執行程式碼，您應該就能點選「啟用攝影機」按鈕，等待網路攝影機載入，然後按住每個資料收集按鈕，收集各類別的資料範例。如您所見，我分別收集了手機和手部的資料。

Make your own "Teachable Machine" using Transfer Learning with MobileNet v3 in TensorFlow.js using saved graph model from TFHub.

MobileNet v3 loaded successfully!



Enable Webcam

Gather Class 1 Data

Gather Class 2 Data

Train & Predict!

Reset

如上方的螢幕截圖所示，狀態文字應會更新，因為系統會將所有張量儲存在記憶體中。

14. 訓練與預測

下一步是為目前空白的 `trainAndPredict()` 函式實作程式碼，這也是執行遷移學習的位置。請看以下程式碼：

script.js

```
async function trainAndPredict() {
  predict = false;
  tf.util.shuffleCombo(trainingDataInputs, trainingDataOutputs);
  let outputsAsTensor = tf.tensor1d(trainingDataOutputs, 'int32');
  let oneHotOutputs = tf.oneHot(outputsAsTensor, CLASS_NAMES.length);
  let inputsAsTensor = tf.stack(trainingDataInputs);

  let results = await model.fit(inputsAsTensor, oneHotOutputs, {shuffle: true, batchSize: 5,
epochs: 10,
  callbacks: {onEpochEnd: logProgress} });

  outputsAsTensor.dispose();
  oneHotOutputs.dispose();
  inputsAsTensor.dispose();
  predict = true;
  predictLoop();
}

function logProgress(epoch, logs) {
  console.log('Data for epoch ' + epoch, logs);
}
```

首先，請將 `predict` 設為 `false`，確保停止目前進行的任何預測。

接著，使用 `tf.util.shuffleCombo()` 隨機排序輸入和輸出陣列，確保順序不會在訓練時造成問題。

將輸出陣列 `trainingDataOutputs`，轉換為 `int32` 類型的 `tensor1d`，以便用於[單一熱編碼](#)。這會儲存在名為 `outputsAsTensor` 的變數中。

請搭配使用 `tf.oneHot()` 函式和這個 `outputsAsTensor` 變數，以及要編碼的類別數量上限 (即 `CLASS_NAMES.length`)。經過 One-Hot 編碼的輸出內容現在會儲存在名為 `oneHotOutputs` 的新張量中。

請注意，目前 `trainingDataInputs` 是記錄張量的陣列。如要使用這些張量陣列進行訓練，您必須將其轉換為一般 2D 張量。

為此，TensorFlow.js 程式庫中提供一個很棒的函式，稱為 `tf.stack()`，

這個函式會接收張量陣列，並將這些張量堆疊在一起，產生維度較高的張量做為輸出。在本例中，系統會傳回 2D 張量，也就是一批長度各為 1024 的 1 維輸入內容，內含記錄的特徵，這正是訓練所需的內容。

接著，`await model.fit()` 訓練自訂模型頭。在這裡，您會傳遞 `inputsAsTensor` 變數和 `oneHotOutputs`，分別代表範例輸入和目標輸出所用的訓練資料。在第 3 個參數的設定物件中，將 `shuffle` 設為 `true`，使用 5 的 `batchSize`，並將 `epochs` 設為 10，然後為 `onEpochEnd` 指定 `callback` 給您稍後定義的 `logProgress` 函式。

最後，由於模型已訓練完成，您可以處置所建立的張量。接著，您可以將 `predict` 設回 `true`，允許再次進行預測，然後呼叫 `predictLoop()` 函式，開始預測即時網路攝影機圖像。

您也可以定義 `logProcess()` 函式來記錄訓練狀態，該函式會用於上述 `model.fit()` 中，並在每輪訓練後將結果列印到控制台。

就快完成了！現在要新增 `predictLoop()` 函式來進行預測。

核心預測迴圈

您可以在這裡實作主要預測迴圈，從網路攝影機取樣影格，並持續預測每個影格的內容，在瀏覽器中即時顯示結果。

請檢查程式碼：

script.js

```
function predictLoop() {
  if (predict) {
    tf.tidy(function() {
      let videoFrameAsTensor = tf.browser.fromPixels(VIDEO).div(255);
      let resizedTensorFrame = tf.image.resizeBilinear(videoFrameAsTensor,
[MOBILE_NET_INPUT_HEIGHT,
      MOBILE_NET_INPUT_WIDTH], true);

      let imageFeatures = mobilenet.predict(resizedTensorFrame.expandDims());
      let prediction = model.predict(imageFeatures).squeeze();
      let highestIndex = prediction.argmax().arraySync();
      let predictionArray = prediction.arraySync();

      STATUS.innerText = 'Prediction: ' + CLASS_NAMES[highestIndex] + ' with ' +
Math.floor(predictionArray[highestIndex] * 100) + '% confidence';
    });

    window.requestAnimationFrame(predictLoop);
  }
}
```

首先，請檢查 `predict` 是否為 `true`，確保只有在模型訓練完成並可供使用後，才會進行預測。

接著，您可以像在 `dataGatherLoop()` 函式中一樣，取得目前圖片的圖片特徵。基本上，您會使用 `tf.browser.fromPixels()` 從網路攝影機擷取影格、將影格標準化、將大小調整為 224 x 224 像素，然後將該資料傳遞至 MobileNet 模型，以取得產生的圖片特徵。

不過，現在您可以傳遞透過訓練模型 `predict()` 函式找到的 `imageFeatures`，使用新訓練的模型頭實際執行預測。接著，您可以擠壓產生的張量，再次將其設為一維，並指派給名為 `prediction` 的變數。

使用這個 `prediction`，您可以使用 `argMax()` 找出具有最高值的索引，然後使用 `arraySync()` 將這個產生的張量轉換為陣列，以取得 JavaScript 中的基礎資料，找出最高值元素的位置。這個值會儲存在名為 `highestIndex` 的變數中。

您也可以直接呼叫 `prediction` 張量上的 `arraySync()`，以相同方式取得實際的預測信賴分數。

現在您已具備所有必要條件，可以利用 `prediction` 資料更新 `STATUS` 文字。如要取得類別的人類可讀字串，只要在 `CLASS_NAMES` 陣列中查閱 `highestIndex`，然後從 `predictionArray` 取得信心值即可。如要以百分比表示，方便閱讀，只要將結果乘以 100 並 `math.floor()` 即可。

最後，準備就緒後，您可以再次使用 `window.requestAnimationFrame()` 呼叫 `predictionLoop()`，即時分類影片串流。如果您選擇使用新資料訓練新模型，這個程序會持續進行，直到 `predict` 設為 `false` 為止。

這就是最後一塊拼圖。實作重設按鈕。

15. 實作重設按鈕

即將完成！最後一塊拼圖是實作重設按鈕，以便重新開始。目前空白的 `reset()` 函式程式碼如下所示。請按照下列方式更新：

script.js

```
/**
 * Purge data and start over. Note this does not dispose of the loaded
 * MobileNet model and MLP head tensors as you will need to reuse
 * them to train a new model.
 */
function reset() {
  predict = false;
  examplesCount.length = 0;
  for (let i = 0; i < trainingDataInputs.length; i++) {
    trainingDataInputs[i].dispose();
  }
  trainingDataInputs.length = 0;
  trainingDataOutputs.length = 0;
  STATUS.innerText = 'No data collected';

  console.log('Tensors in memory: ' + tf.memory().numTensors);
}
```

首先，請將 `predict` 設為 `false`，停止所有執行中的預測迴圈。接著，將 `examplesCount` 陣列的長度設為 0，即可刪除陣列中的所有內容。這是清除陣列中所有內容的實用方法。

現在請檢查所有目前記錄的 `trainingDataInputs`，並確保您 `dispose()` 其中包含的每個張量，再次釋放記憶體，因為 JavaScript 垃圾收集器不會清理張量。

完成後，您現在可以安全地將 `trainingDataInputs` 和 `trainingDataOutputs` 陣列的長度設為 0，藉此清除這些陣列。

注意：如果您在處置張量前，已將 `trainingDataInputs` 陣列的長度設為零，張量會無法存取，但仍會保留在記憶體中，且不會遭到處置，這會導致記憶體流失。

最後，將 `STATUS` 文字設為有意義的內容，並列印記憶體中剩餘的張量，做為健全性檢查。

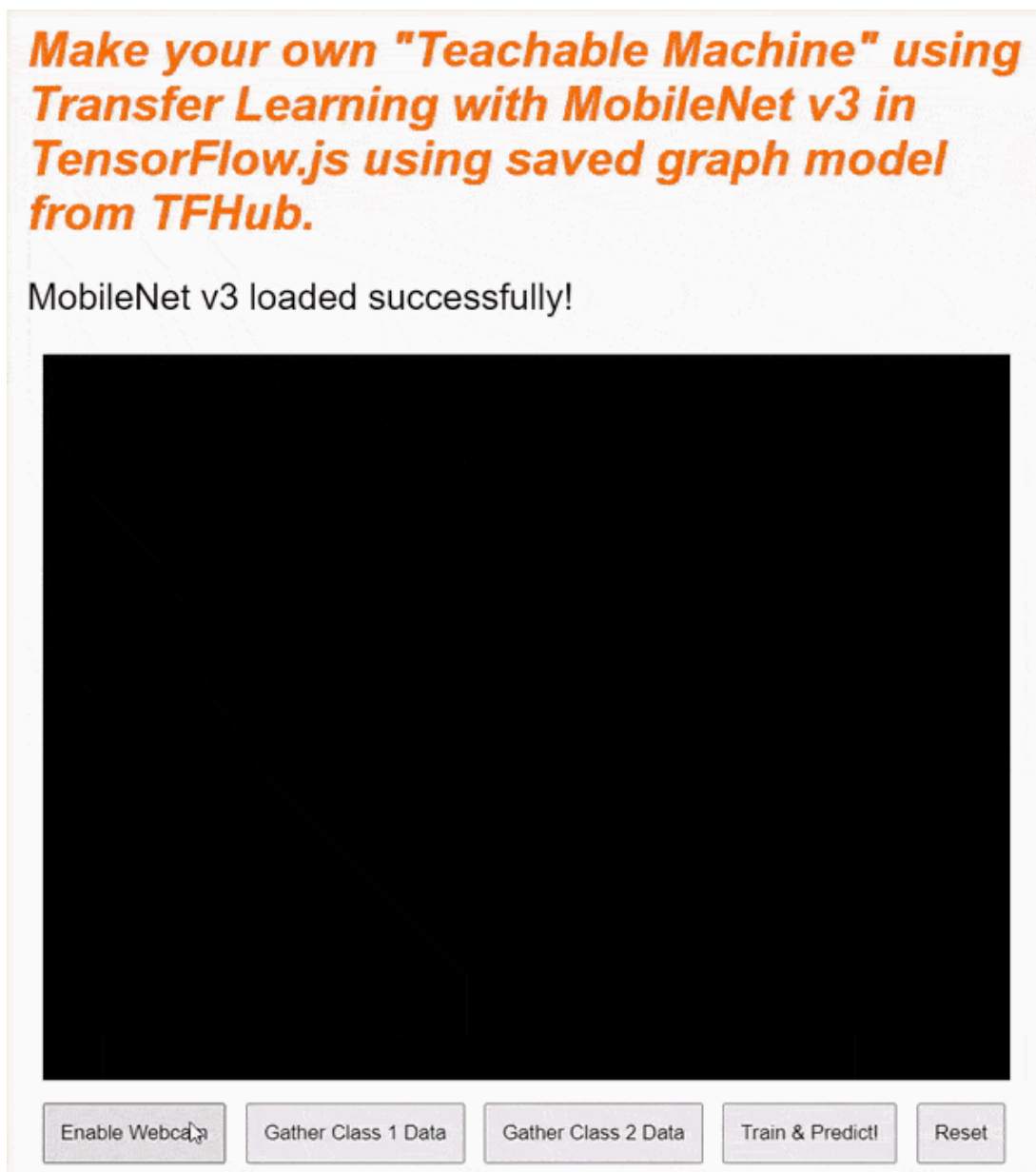
請注意，由於您定義的 MobileNet 模型和多層感知器都未處置，因此記憶體中仍有數百個張量。如果重設後決定再次訓練模型，就必須使用新的訓練資料。

步驟 16 · 約 5 分鐘

16. 試試看吧！

現在來測試您自己的 Teachable Machine 版本吧！

前往即時預覽畫面，啟用網路攝影機，為房間中的某個物件收集至少 30 個類別 1 的樣本，然後為另一個物件收集類別 2 的樣本，按一下「訓練」，並查看控制台記錄，瞭解進度。訓練速度應該很快：



訓練完成後，將物件對準攝影機，即可取得即時預測結果，並顯示在網頁頂端附近的狀態文字區域。如有任何問題，請[查看我完成的有效程式碼](#)，確認您是否遺漏任何複製內容。

17. 恭喜

恭喜！您剛才使用 TensorFlow.js 在瀏覽器中，完成了第一個遷移學習範例。

請試試這項功能，並測試各種物體。你可能會發現，部分物體比其他物體更難辨識，尤其是與其他物體相似的物體。您可能需要新增更多類別或訓練資料，才能區分兩者。

重點回顧

在本程式碼研究室中，您已瞭解：

1. 遷移學習的定義，以及相較於訓練完整模型的優勢。
2. 如何從 TensorFlow Hub 取得可重複使用的模型。
3. 如何設定適合遷移學習的網頁應用程式。
4. 如何載入及使用基礎模型來生成圖片特徵。
5. 如何訓練新的預測頭，以便從網路攝影機影像中辨識自訂物件。
6. 如何使用產生的模型即時分類資料。

後續步驟

您現在已具備可供著手進行的基礎，接下來可以發想哪些創意，擴充這個機器學習模型樣板，以用於您可能正在處理的實際應用情境？或許您可以革新目前所屬的產業，協助貴公司訓練模型，分類日常工作中重要的事物？一切都有無限的可能。

如要進一步瞭解，[不妨免費修讀這門完整課程](#)，瞭解如何將本程式碼研究室中的 2 個模型合併為 1 個模型，提升效率。

此外，如要進一步瞭解原始 Teachable Machine 應用程式背後的理論，請參閱[這篇教學課程](#)。

與我們分享你的作品

您也可以輕鬆將今天製作的內容用於其他創意用途，我們鼓勵您發揮創意，持續探索各種可能性。

別忘了在社群媒體上使用 [#MadeWithTFJS](#) 主題標記，你的專案就有機會登上 [TensorFlow 網誌](#)，甚至在 [日後活動](#) 中展示。我們很期待看到你的作品。

值得一訪的網站

- [TensorFlow.js 官方網站](#)
 - [TensorFlow.js 預先建構模型](#)
 - [TensorFlow.js API](#)
 - [TensorFlow.js 展演秀](#)：從他人作品中汲取靈感。
-